

COSpark

Project report submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences



Submitted by

Kadiyala Jaya Lahari AP23110011034

Paturi Bhavana AP23110010823

Naga Abhinay Kota AP23110010999

Peddireddi Venkata Saketh Ram AP23110010955

INTRODUCTION

CoSpark is a modern and user-friendly web-based application designed to connect aspiring entrepreneurs, developers, designers, and professionals who are looking to build startups collaboratively. The platform aims to solve one of the most common problems in the startup ecosystem: finding the right co-founder with complementary skills.

CoSpark allows users to create professional profiles, list their skills, and explore startup ideas posted by others. Users can also post their own startup ideas and specify the required roles, such as developer, designer, or marketer.

The platform provides a structured collaboration environment where users can send and receive requests to join startup teams. With features like skill-based filtering, idea exploration, and collaboration management, CoSpark simplifies the process of transforming ideas into real-world startups.

It is built using modern full-stack technologies such as React.js for frontend development, Node.js and Express.js for backend services, and MongoDB for database management, ensuring scalability, efficiency, and smooth user interaction.

SCENARIO BASED CASE STUDY

Meet Geeta, a passionate developer who has multiple innovative startup ideas but lacks a strong team to bring them to life. She finds it difficult to connect with designers and marketers who share her vision. Geeta discovers CoSpark, a co-founder matching platform designed to help individuals collaborate on startup ideas.

Geeta registers on the platform and securely logs in using her credentials. After logging in, she creates her profile by adding her role as a developer and listing her skills such as React.js and Node.js. She then posts a startup idea titled “AI Resume Builder” and specifies required roles such as UI/UX Designer and Marketing Specialist.

Meanwhile, another user explores ideas on the platform and discovers Geeta's idea. Interested in collaborating, the user sends a request to join her startup as a designer. Geeta reviews the applicant's profile and accepts the request, leading to the formation of a startup team.

Through CoSpark, Geeta successfully transforms her idea into execution by collaborating with the right people, demonstrating how the platform enables efficient co-founder matching and teamwork.

SYSTEM REQUIREMENTS:

To ensure smooth development, deployment, and usage of the CoSpark web-Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable agricultural platform.

a)Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- Operating System: Windows 10/11, macOS, or Linux
- Node.js (v16 or above): Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing.
- React.js: A JavaScript library for building dynamic and responsive user interfaces.
- Browser: Google Chrome / Firefox — For rendering and testing the UI in real-time.
- Express.js: A lightweight web framework for building RESTful APIs.
- MongoDB: A NoSQL database used to store structured and unstructured data related to farms, users, and resources.
- Postman: Tool for testing APIs during development.
- Visual Studio Code: Preferred code editor with built-in Git and terminal support.

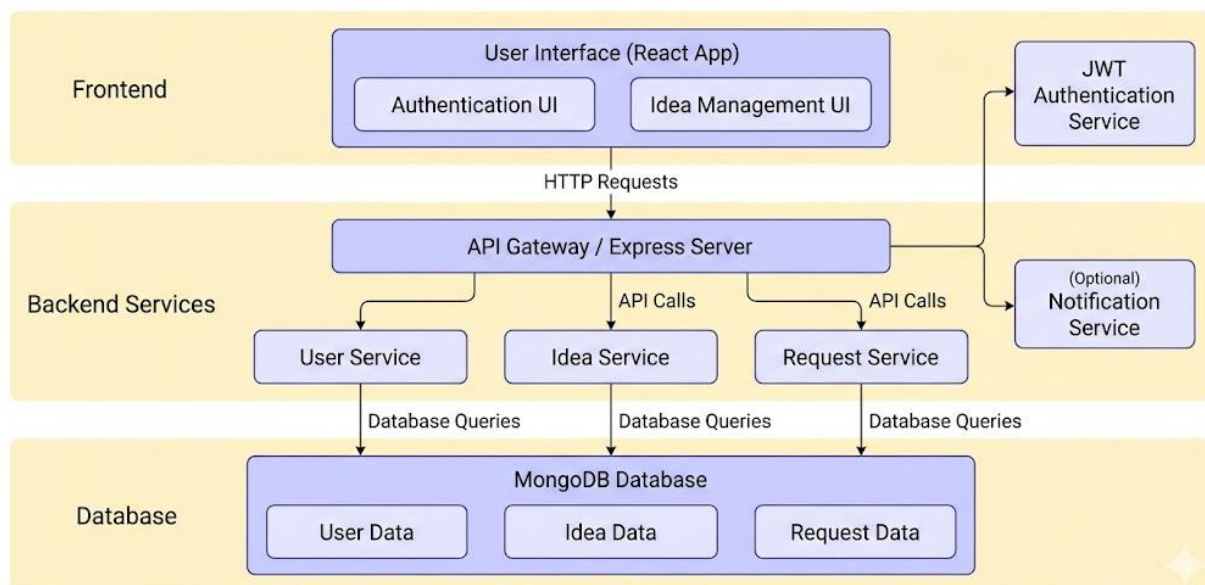
b) Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better. Ensures fast compilation and multitasking during development.
- RAM: Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- Storage: At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- Display: 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

PROJECT ARCHITECTURE

TECHNICAL ARCHITECTURE:



The CoSpark platform follows a **client-server architecture** that ensures efficient communication between the user interface, backend logic, and database management system. This architecture is designed to provide scalability, maintainability, and smooth user interaction.

In this architecture, the process begins when a user interacts with the CoSpark frontend interface. The frontend is developed using React.js and provides an interactive environment where users can perform various actions such as registration, login, profile creation, posting startup ideas, exploring ideas, and sending collaboration requests.

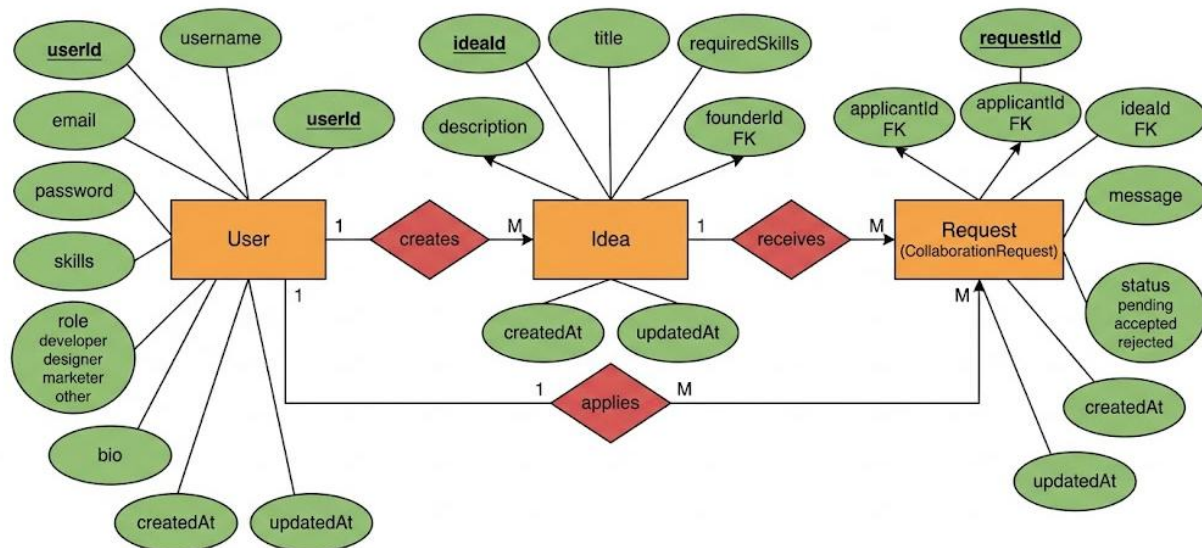
When a user performs any action, the frontend sends an HTTP request to the backend server through APIs. These requests are handled using Axios or Fetch, ensuring seamless communication between the client and server.

The backend, built using Node.js and Express.js, processes these requests by executing the appropriate business logic. This includes validating user input, authenticating users using JWT (JSON Web Tokens), handling CRUD operations, and managing collaboration workflows.

The backend server communicates with the MongoDB database using Mongoose. The database stores all essential data such as user profiles, startup ideas, required skills, and collaboration requests. MongoDB's flexible schema structure allows efficient handling of dynamic data.

Once the backend completes processing the request, it sends a response back to the frontend. The frontend then updates the user interface dynamically, providing real-time feedback to the user.

ER DIAGRAM:



The Entity Relationship (ER) diagram of CoSpark visually represents the structure of the database and the relationships between different entities in the system. It helps in understanding how data is stored, connected, and accessed within the platform.

Entities:

1. User

The User entity represents individuals registered on the platform. Each user can act as a founder or a co-founder.

Attributes:

- userId (Primary Key)
- username
- email
- password
- role (Developer, Designer, Marketer, etc.)

- skills (array of skills)
- bio (optional description)
- createdAt

2. Idea

The Idea entity represents startup ideas posted by users.

Attributes:

- ideaId (Primary Key)
- title
- description
- founderId (Foreign Key referencing User)
- requiredSkills
- createdAt

3. Request

The Request entity represents collaboration requests sent by users to join startup ideas.

Attributes:

- requestId (Primary Key)
- ideaId (Foreign Key referencing Idea)
- applicantId (Foreign Key referencing User)
- status (Pending / Accepted / Rejected)
- message (optional)
- createdAt

Relationships:

- A **User** can create **multiple Ideas** (One-to-Many relationship)
- A **User** can send **multiple Requests** (One-to-Many relationship)
- An **Idea** can receive **multiple Requests** (One-to-Many relationship)

These relationships ensure that multiple users can collaborate on a single idea while maintaining proper data structure.

Keys:

Primary Keys:

- `userId`
- `ideaId`
- `requestId`

Foreign Keys:

- `idea.founderId` → `user.userId`
- `request.ideaId` → `idea.ideaId`
- `request.applicantId` → `user.userId`

These keys maintain referential integrity and ensure consistency in the database.

KEY FEATURES

The CoSpark platform provides a wide range of features that enhance user experience and enable efficient collaboration among startup enthusiasts. The system is designed to simplify the process of connecting individuals with complementary skills and transforming ideas into collaborative startup projects.

The platform offers secure user registration and authentication functionality using JWT (JSON Web Tokens). Users can create accounts and log in safely, while passwords are encrypted using bcrypt to ensure data security and protection against unauthorized access.

CoSpark also includes a comprehensive profile management system that allows users to create and maintain their profiles. Users can add relevant details such as skills, roles (developer, designer, marketer, etc.), and personal information. This structured profile system helps in better matching users with suitable startup ideas and collaboration opportunities.

Another important feature is startup idea posting and management. Users can post their ideas by providing a title, description, and required skills. They also have the flexibility to update or delete their ideas, ensuring that the information remains accurate and up to date.

The platform supports skill-based co-founder matching, which allows users to discover opportunities that align with their expertise and interests. This feature significantly improves the chances of forming effective and well-balanced startup teams.

CoSpark also includes a collaboration request system where users can send requests to join startup ideas. Founders can review these requests and accept or reject them based on the applicant's skills and suitability. This ensures a structured and controlled collaboration process.

Additionally, users can explore startup ideas using filtering options based on skills, roles, and interests. This makes it easier for users to find relevant ideas quickly and efficiently.

The frontend of the platform is designed with modern UI/UX principles and follows a responsive design approach. This ensures that the application is accessible and user-friendly across different devices, including desktops, tablets, and mobile phones.

ROLES AND RESPONSIBILITIES

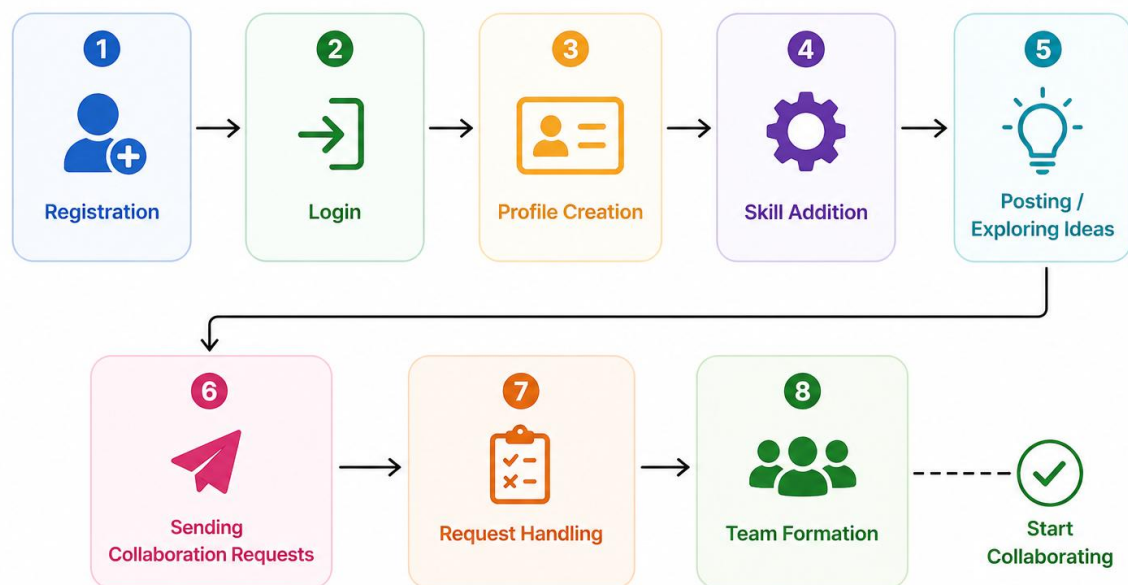
User:

- **Registration and Account Management:** Users are responsible for creating an account by providing accurate personal and professional information. They must manage their account details, including profile updates, password security, and maintaining the confidentiality of their login credentials.
- **Profile Creation and Skill Management:** Users are required to create and maintain a detailed profile by adding their skills, roles (such as Developer, Designer, or Marketer), and other relevant information. This helps in improving visibility and matching with suitable startup opportunities.
- **Startup Idea Posting:** Users can post their startup ideas by providing a clear title, description, and required skills. They are responsible for ensuring that the information provided is accurate and useful for potential collaborators.
- **Idea Exploration and Search:** Users can browse and search for startup ideas based on their interests, skills, and roles. They are responsible for selecting ideas that align with their expertise and goals.
- **Collaboration Requests:** Users can send requests to join startup ideas by expressing their interest and suitability. They must ensure that their requests are relevant and professional.
- **Request Management:** Users who post ideas (founders) are responsible for reviewing incoming collaboration requests. They can accept suitable candidates or reject requests based on requirements.
- **Communication and Collaboration:** Users are expected to communicate effectively with team members after joining a startup idea. Proper collaboration ensures successful execution of the project.
- **Feedback and Platform Usage:** Users should use the platform responsibly and may provide feedback to improve the system. They should avoid misuse and follow platform guidelines.

Admin:

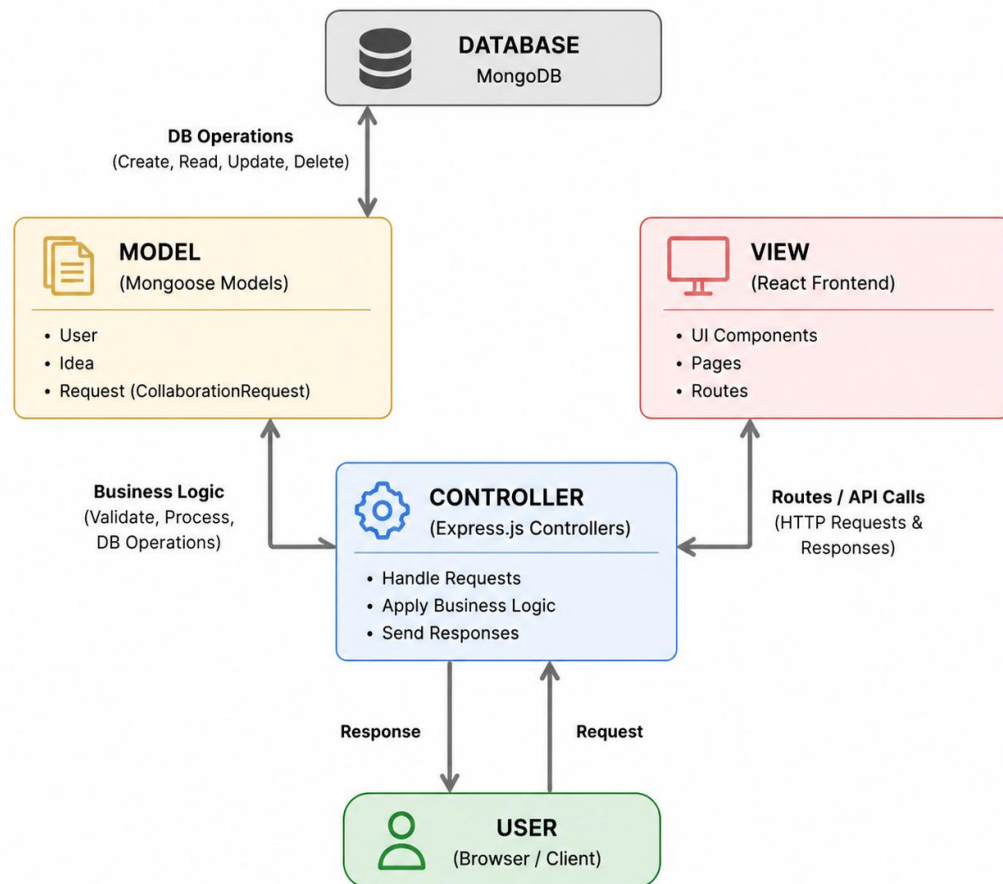
- **System Management:** The admin is responsible for the overall management and smooth functioning of the CoSpark platform. This includes monitoring system performance, maintaining server stability, and ensuring efficient operation of the application.
- **User Management:** The admin manages user accounts, including registration verification, account monitoring, and handling issues such as suspicious activity, account suspension, or removal when necessary.
- **Content Monitoring:** The admin ensures that all posted startup ideas and user-generated content are appropriate, relevant, and adhere to platform guidelines. Any inappropriate or misleading content is removed.
- **Security and Data Management:** The admin ensures data security, privacy, and protection of user information. They monitor the system for potential security threats and take necessary actions.
- **Platform Maintenance:** The admin is responsible for updating the system, fixing bugs, and implementing improvements to enhance user experience.

User Flow:



- **Registration:** The process begins when a user signs up on the platform by providing basic details such as email and password to create an account.
- **Login:** After registration, the user logs into the system using valid credentials. Authentication is handled securely using JWT.
- **Profile Creation:** The user creates a profile by adding essential details such as skills, role, and personal information.
- **Skill Addition:** Users specify their skills, which helps in matching them with relevant startup ideas and collaboration opportunities.
- **Posting or Exploring Ideas:** Users can either post a new startup idea or explore ideas shared by other users based on their interests.
- **Sending Collaboration Requests:** Users can send requests to join startup ideas that match their skills and interests.
- **Request Handling:** The idea owner reviews incoming requests and decides to accept or reject them based on suitability.
- **Team Formation:** Once accepted, users become part of the startup team and can collaborate further on the project.

MVC PATTERN



The CoSpark application follows the **Model-View-Controller (MVC) architectural pattern**, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability of the system.

Model Layer:

The Model layer is responsible for handling all data-related logic in the CoSpark application. This includes defining data schemas and performing operations on the database using those schemas.

The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

In CoSpark, the main models include:

- User model (stores user details, skills, roles)
- Idea model (stores startup ideas)
- Request model (stores collaboration requests)

Controller Layer:

The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests from the frontend, processes the input (including validation and authentication), and calls the appropriate methods from the model.

After processing, it sends the appropriate response back to the client.

In CoSpark, controllers handle:

- User authentication (login, signup)
- Idea management (create, update, delete)
- Collaboration request handling

View Layer:

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined.

These endpoints determine how the backend responds to different HTTP requests such as:

- GET
- POST
- PUT
- DELETE

The routes are responsible for invoking the appropriate controller functions and returning responses to the frontend.

Advantages:

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features can be added easily by creating new routes, controllers, and models.
- **Reusability:** Logic in controllers and models can be reused across multiple parts of the application.
- **Testing:** Each layer can be tested independently, especially the controllers and models.
- **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers without conflict.

PROJECT SETUP AND CONFIGURATION

Creating Project Folder:

1. Create a new folder with the project name **CoSpark**.
2. Inside the main folder, create two subfolders.
3. Name one folder as **Client**.
4. Name another folder as **Server**.
5. Open the project folder in **Visual Studio Code** for development.

Client Setup (Installing React App):

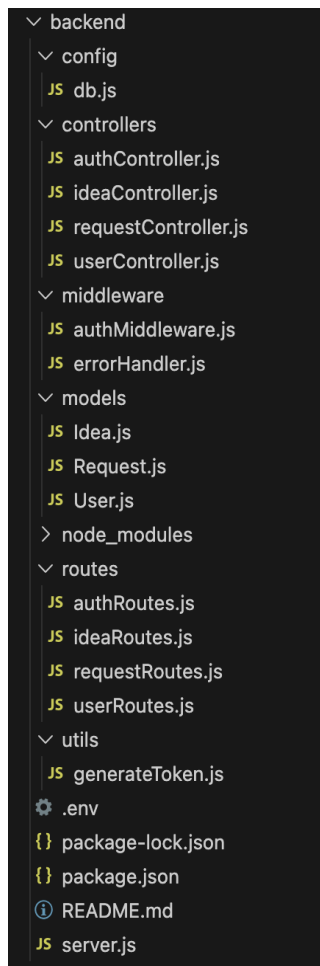
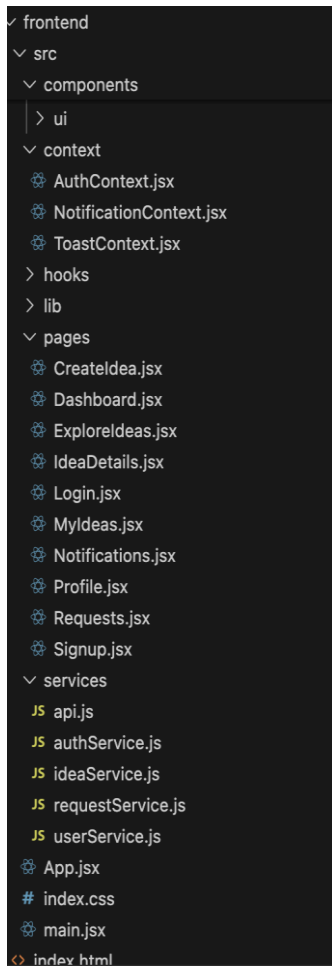
- Open the Client folder in the terminal of VS Code.
- Run the following command to create a React application using Vite:
`npm create vite@latest . -- --template react`
- Select React framework from the given options:
 - Select a framework:
 - React

- Select JavaScript variant from the given options:
 - Select a variant:
 - JavaScript
- Navigate to the client folder:
 - cd client
- Install all required dependencies:
 - npm install
- Start the React development server:
 - npm run dev

Server Setup (npm init):

- Open the Server folder in the terminal of VS Code.
 - Initialize Node.js project:
 - npm init -y
 - Create main file:
 - server.js
 - Create the following folders:
 - models
 - controllers
 - routes
 - middleware

PROJECT STRUCTURE:



BACKEND DEVELOPMENT:

The backend of the CoSpark application is developed using Node.js and Express.js, which provides a robust environment for handling server-side logic and API communication.

Controllers:

- `authController.js` → Handles user authentication (login, signup, JWT token generation).
- `ideaController.js` → Handles idea-related operations (create, update, delete, fetch ideas).

- requestController.js → Manages collaboration requests (send, accept, reject).
- userController.js → Handles user profile operations (view and update profile details).

Models:

- User.js → Stores user details such as skills, role, and profile data.
- Idea.js → Stores startup ideas posted by users.
- Request.js → Stores collaboration request details.

Middleware:

- authMiddleware.js → Handles authentication and protects routes.
- errorHandler.js → Handles errors and API responses.

Routes:

- authRoutes.js → Routes for user authentication (login and registration).
- ideaRoutes.js → Routes for idea management (create, update, delete, fetch).
- requestRoutes.js → Routes for collaboration requests (send, accept, reject).
- userRoutes.js → Routes for user profile management.

Backend Functionality:

- Build RESTful APIs using Express.js.
- Implement JWT authentication for secure user access.
- Use middleware for authentication and error handling.
- Perform CRUD operations:
 - Create → Add users, ideas, requests
 - Read → Fetch data
 - Update → Modify data
 - Delete → Remove data
- Connect the backend with MongoDB using Mongoose for database operations.

DATABASE DEVELOPMENT:

- Use MongoDB with Mongoose
- Create schemas based on ER diagram
- Store:
 - Users
 - Ideas
 - Requests

FRONTEND DEVELOPMENT:

Components (src/components):

layout:

- Layout.jsx → Defines the overall layout structure of the application including page arrangement.
- Navbar.jsx → Navigation bar used across the application for routing between pages.

shared:

- DeleteConfirmDialog.jsx → Displays confirmation dialog before deleting an idea or action.
- EditIdeaModal.jsx → Modal component used to edit existing startup ideas.
- EmptyState.jsx → Displays a message or UI when no data is available.
- FormInput.jsx → Reusable input component used in forms across the application.
- IdeaCard.jsx → Displays individual startup idea details in card format.
- Loader.jsx → Loading spinner component shown during API calls or processing.
- Profile.jsx → Reusable component for displaying user profile details.
- ProtectedRoute.jsx → Restricts access to certain routes for authenticated users only.
- SkillBadge.jsx → Displays individual skills as badges/tags.
- SkillSelector.jsx → Component for selecting and managing user skills.

ui:

- ui/ → Contains reusable UI elements such as buttons, inputs, modals, and styled components used throughout the application.

Context (src/context):

- AuthContext.jsx → Manages user authentication state and session handling across the application.
- NotificationContext.jsx → Handles real-time notifications related to collaboration requests and updates.
- ToastContext.jsx → Manages toast messages for alerts, success messages, and error feedback.

Pages (src/pages):

- CreateIdea.jsx → UI for creating and posting new startup ideas.
- Dashboard.jsx → Main dashboard showing user activity and overview.
- ExploreIdeas.jsx → Page for browsing and discovering startup ideas.
- IdeaDetails.jsx → Displays detailed information about a selected idea.
- Login.jsx → User login page for authentication.
- MyIdeas.jsx → Displays ideas created by the logged-in user.
- Notifications.jsx → Shows user notifications related to requests and updates.
- Profile.jsx → User profile page for viewing and editing personal details.
- Requests.jsx → Displays collaboration requests (sent and received).
- Signup.jsx → User registration page.

Services (src/services):

- api.js → Configures API base URL and handles HTTP requests.
- authService.js → Handles authentication-related API calls (login, signup).
- ideaService.js → Handles API operations related to startup ideas.
- requestService.js → Manages collaboration request APIs.
- userService.js → Handles user profile-related API operations.

Main Files:

- App.jsx → Main component that manages routing and layout of the application.
- main.jsx → Entry point of the React application that renders the app.
- index.css → Global styling for the application.
- index.html → Root HTML file where the React app is mounted.

OUTPUT:

- Signup Page

**CoSpark**

Start your startup journey today

- 01 Create your profile
- 02 Post or explore ideas
- 03 Find your co-founder

Create your account

Join thousands of founders building the future

Username

Email address

Password

What do you bring?

Skills (optional)

Browse by category or search — press Enter to add a custom skill

Create Account →

Already have an account? [Sign in](#)

- Login Page

**CoFoundr**

Find your perfect co-founder

Match with talented builders who share your vision and complement your skills.

- Instant skill matching
- 500+ active founders
- 200+ ideas launched

Welcome back

Sign in to continue building your startup

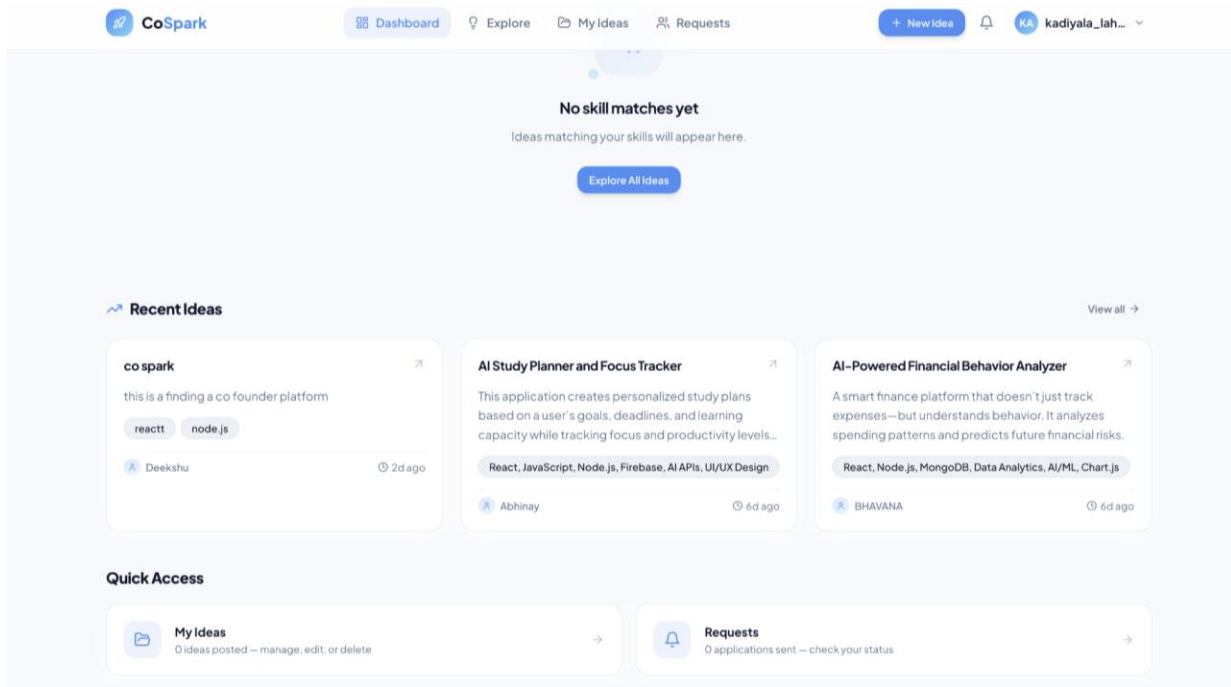
Email address

Password

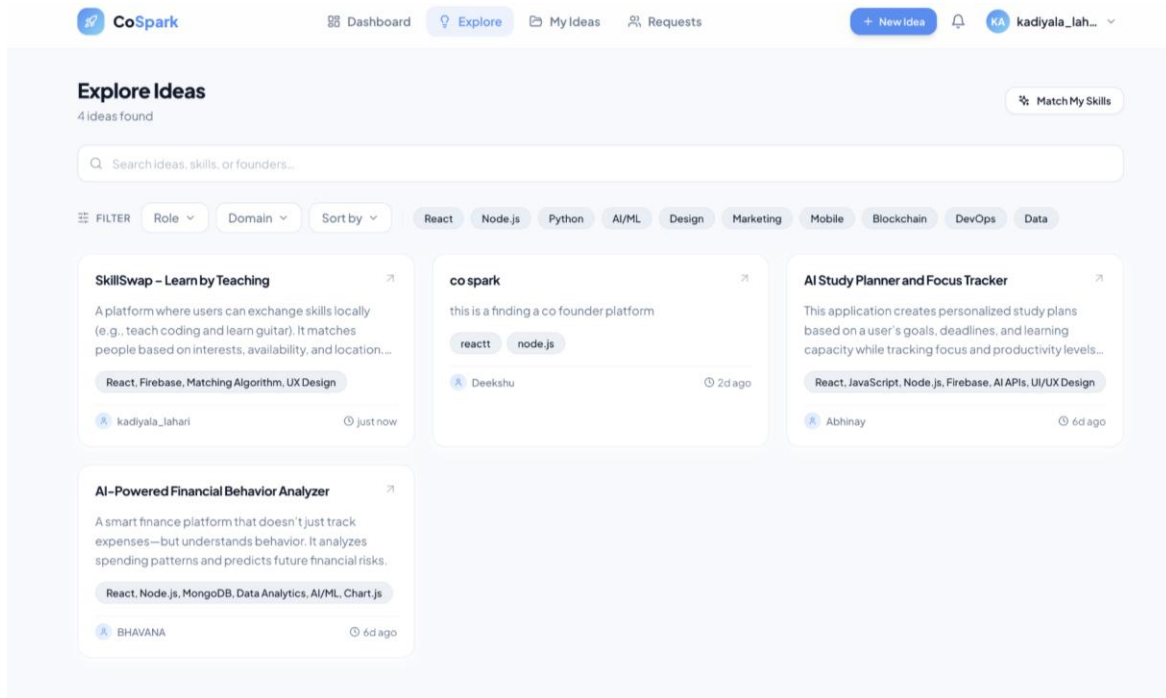
Sign In →

No account? [Create one](#) — it's free

- Dashboard



- Explore Page



- Idea Page

CoSpark

Dashboard

Explore

My Ideas

Requests

+ New Idea

KA kadiyala_lah...

← Back

Post a Startup Idea

Share your vision and find the perfect co-founder

Idea Details

Give your idea a clear, compelling description.

Idea Title

Local Skill Exchange Platform

Description

218/2000

A platform where users can exchange skills locally (e.g., teach coding and learn guitar). It matches people based on interests, availability, and location. The aim is to make learning accessible without money barriers.

Required Skills (optional)

React, Firebase, Matching Algorithm, UX Design

Press Enter or comma to add a skill
Helps match your idea with the right founders.

Cancel

Post Idea

- Profile Page

CoSpark

Dashboard

Explore

My Ideas

Requests

+ New Idea

KA kadiyala_lah...

KA

Edit Profile

kadiyala_lahari

Developer

kadiyalajalahari@gmail.com

No bio yet — add one!

Skills

Next.js

Vue

Angular

Flutter

Account Details

Email

kadiyalajalahari@gmail.com

Member since

April 2026

Account status

Active

Posted Ideas

(1)

View all →

Github Link :<https://github.com/Bhavana105802/co-spark->

Demo Video Link: [https://drive.google.com/file/d/1H-nUlaon6KJcQj1ADPOaW4V3kxac_zSk/view?usp=drive link](https://drive.google.com/file/d/1H-nUlaon6KJcQj1ADPOaW4V3kxac_zSk/view?usp=drive_link)

Code Drive

Link:[https://drive.google.com/file/d/1EARtMxyMpsSsh09xYFs_i7x968zUCB_Xf/view?usp=drive link](https://drive.google.com/file/d/1EARtMxyMpsSsh09xYFs_i7x968zUCB_Xf/view?usp=drive_link)